Smart Library

Management System

Saraswata Tarafdar
Department: CSE(IOTCSBT)
Roll no. :62

Term-II Individual Project | Spring 2026

Problem: Manual library management is error-prone — no tracking of borrow records, fines, or availability.

Target: Librarians, members, and administrators

2 System Design

model/

LibraryItem, Book, Magazine, Member, BorrowRecord

factory/

LibraryItemFactory [Factory Pattern]

service/

LibraryService [Singleton] + DataManager

exception/

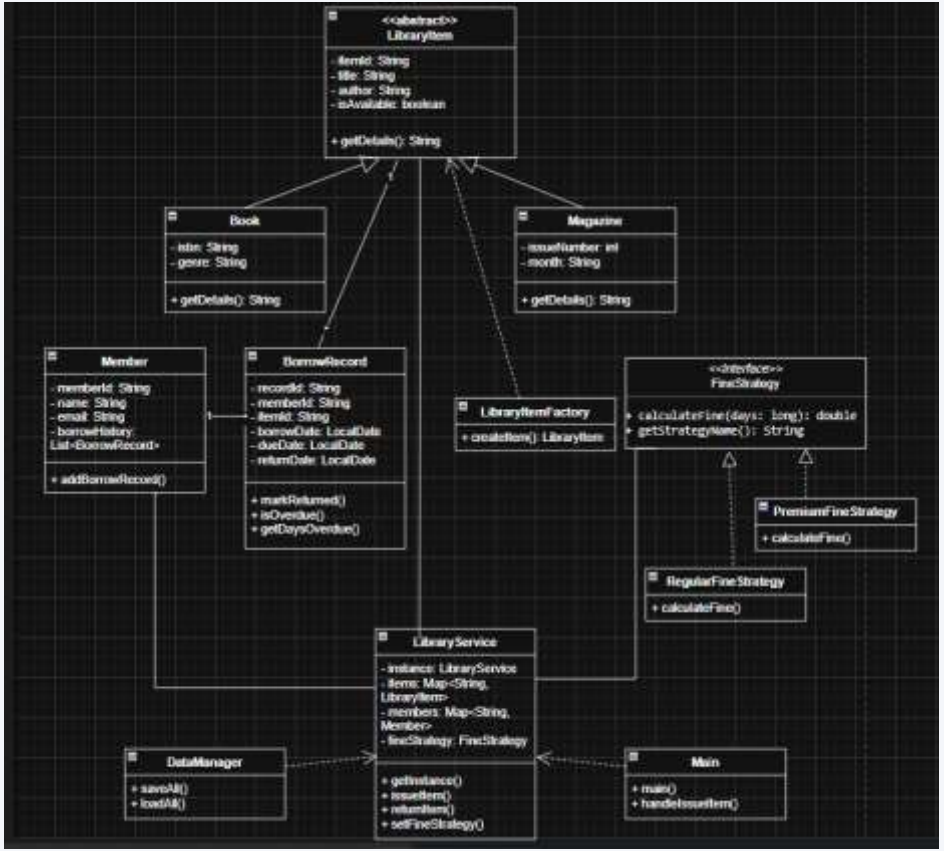
BookNotFoundException, AlreadyBorrowedException, MemberNotFoundException

strategy/

FineStrategy interface + Regular / Premium impls

ui/

Main.java — 12-option console menu



3 OOP Concepts Demonstrated

A Abstraction

LibraryItem (abstract class)

abstract getDetails() forces every subclass to define its own output

I Inheritance

Book, Magazine extend LibraryItem

super() reuses parent fields — no code duplication

P Polymorphism

getDetails() in Book vs Magazine

Same method name, different output at runtime via @Override

E Exception Handling

3 custom exceptions + try-catch

BookNotFoundException carries the exact bad ID for clear feedback

C Collections

HashMap in LibraryService

O(1) lookup by item/member ID — no loop needed

F File Handling

DataManager (Serialization)

ObjectOutputStream writes full HashMap to .ser binary file

4 Technical Highlight — Strategy Pattern + Fine Calculation

Why Strategy Pattern?

- Swap fine algorithm at RUNTIME without changing LibraryService
- Regular: Rs. 2.00 per day — no cap
- Premium: Rs. 1.00 per day — capped at Rs. 30
- Adding a new member type only needs a new class — no existing code changes
- Instructor feedback requested an extra design pattern — this was the chosen solution

FineStrategy.java + usage in LibraryService

```java
public interface FineStrategy {
    double calculateFine(long daysOverdue);
}

public double calculateFine(long days) {
    return days <= 0 ? 0.0 : days * 2.0;
}

public double calculateFine(long days) {
    return Math.min(days * 1.0, 30.0);
}

service.setFineStrategy(
    new PremiumFineStrategy());
```

5 Live Demo

01 Working System

- Program starts → DataManager.loadAll() restores saved data
- Add a Book via Factory: createItem('BOOK', 'B01', ...)
- Add a Member: addMember('M01', 'Saraswata', ...)
- List all items and members — confirm stored in HashMap

02 Meaningful Scenario

- Issue book B01 to member M01 → BorrowRecord created
- Show: book is now unavailable (isAvailable = false)
- Try issuing same book to M02 → AlreadyBorrowedException shown
- Return book M01 → fine calculated via FineStrategy

03 Error Handling

- Search for item with invalid ID → BookNotFoundException
- Try issuing already-borrowed book → AlreadyBorrowedException
- Exit (option 12) → DataManager.saveAll() persists to .ser files
- Restart → data reloaded — persistence confirmed